Stage 0: Press Release - Project description

What system are you building?

This project focuses on building a sports notification application that delivers real time score updates and game events to users through a mobile app. The system allows users to follow specific teams, leagues, or games and receive timely notifications when important events occur such as score changes, game start/end, and any important news.

Why are you building it?

Sports fans often want to stay updated on games they care about but are not always able to watch games live or continuously check scores. Constantly refreshing sports apps or websites can be inconvenient, and many existing platforms overwhelm users with updates they are not interested in. This system is being built to provide users with customizable, relevant, real-time updates so they can stay informed without needing to actively monitor games throughout the day.

Who are you building it for?

- Sports fans who want live score updates and game notifications
- User who follow specific teams, leagues or individual games
- Sports media platforms seeking to deliver live updates to a large user base

What problems are these customers facing and how will your system solve them?

Sports fans often miss important moments in games because they are not watching live or do not have time to repeatedly check scores. Refreshing apps or websites throughout the day is inefficient, and many users receive notifications that are irrelevant to the teams or games they care about. This can lead to missed updates or unnecessary distractions .

This system solves these problems by allowing users to subscribe only to the teams or games they are interested in and receive push notifications when relevant events occur. By delivering targeted, real time updates through a mobile app, the system ensures users stay informed while minimizing unnecessary notifications. The system also saves users time and can reduce battery usage by limiting the need to repeatedly open the app and refresh scores, since updates are delivered through notifications as they happen.

How the system is used

A user opens the mobile app and selects teams, leagues, or games that they want to follow. As game progress and important events occur, the system sends notifications directly to the user. This allows users to stay informed without needing to constantly check on a game. There will be many different notification options to choose from. Users can customize their notification preferences by choosing which types of events they want to be updated about, such as specific news in a league or team, variety of single game alerts and can select specific events they want to be notified about, team specific updates, or team news.

STAGE 1: Service functional/behavioral requirements and initial list of components

Functional and Behavioral requirements

The system must…

- Allow users to select teams, leagues or individual games to follow.
- Allow users to configure notification preferences by toggling alert types on or off.
- Send push notifications when relevant game events or news occur.
- Allow users to enable alerts for a single game or all games of a selected team.
- Maintain the current state of each game.
- Ensure notifications are sent only to users subscribed to that event.
- Store user preferences so they are saved over time
- Support concurrent games and concurrent users

Key Components and Responsibilities

- **Application (client) :** Allows users to create accounts, select teams and manage notification preferences. User Interface for viewing live scores and history.
- **Event Ingestion Service:** Communicates with sports data provider to receive live updates on games
- **Event Processing Service:** Processes incoming events and determines which users should receive notifications based on the users subscription and notification settings.
- **User Preference Service:** Stores user subscription data and notification settings. Maintains which teams, leagues, or games each user follows, as well as specific alert settings they have enabled.
- **Notification Service:** Sends push notifications to user device
- **Game State Management Service:** Maintains live games, including live scores, game status, and game history. This allows the system to provide up to date data for any supported game at any time.

Happy path

1) A user selects teams or games to follow and enables notifications for specific events.
2) A score change occurs during the game.
3) The system receives the event from the sports data provider.
4) The system identifies users who subscribed to that team or game
5) The system then sends notifications to the appropriate users.
6) The user receives the notification and can open the app to view more details

Failure scenarios

- **Sports data unavailable**
    - If the live sports updates are temporarily unavailable, the system should retry periodically and continue delivering notifications until a new connection is

established. During a period of disconnection, previous game information remains available.

- **Duplicate notifications**
    - If the system receives the same event multiple times, it should avoid sending duplicate notifications when possible. Occasional duplicates should not disrupt the system functionality.

- **Notification delivery failure**
    - The system should not only send notifications to users on the app or on their phones at the time a new update is received. If the phone is offline or idle, the notification should go to the user's notification history.

Figure 1 - **System Context Diagram**

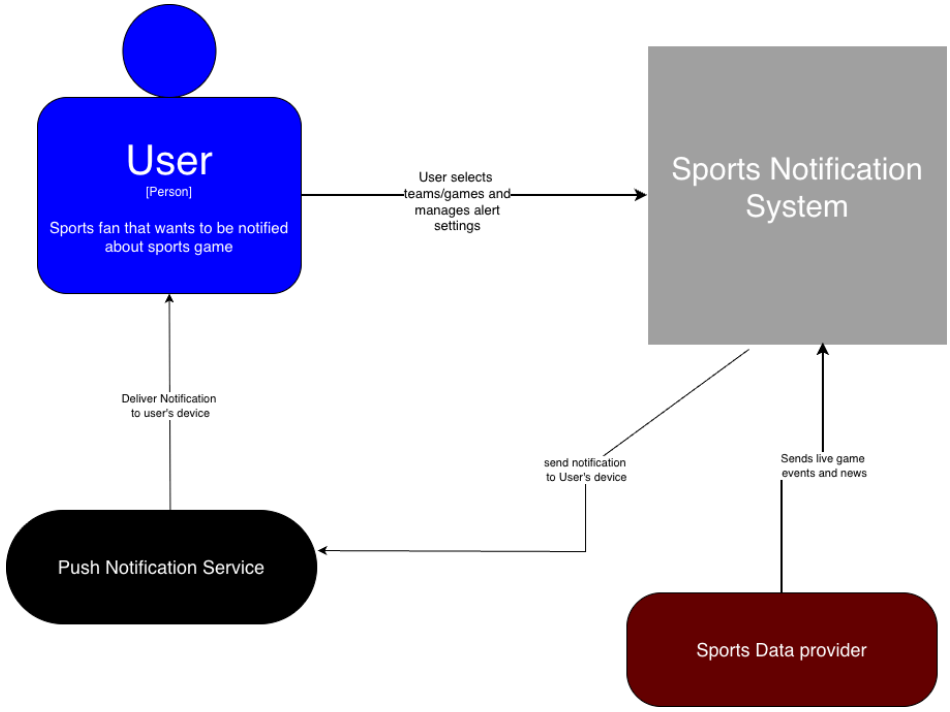
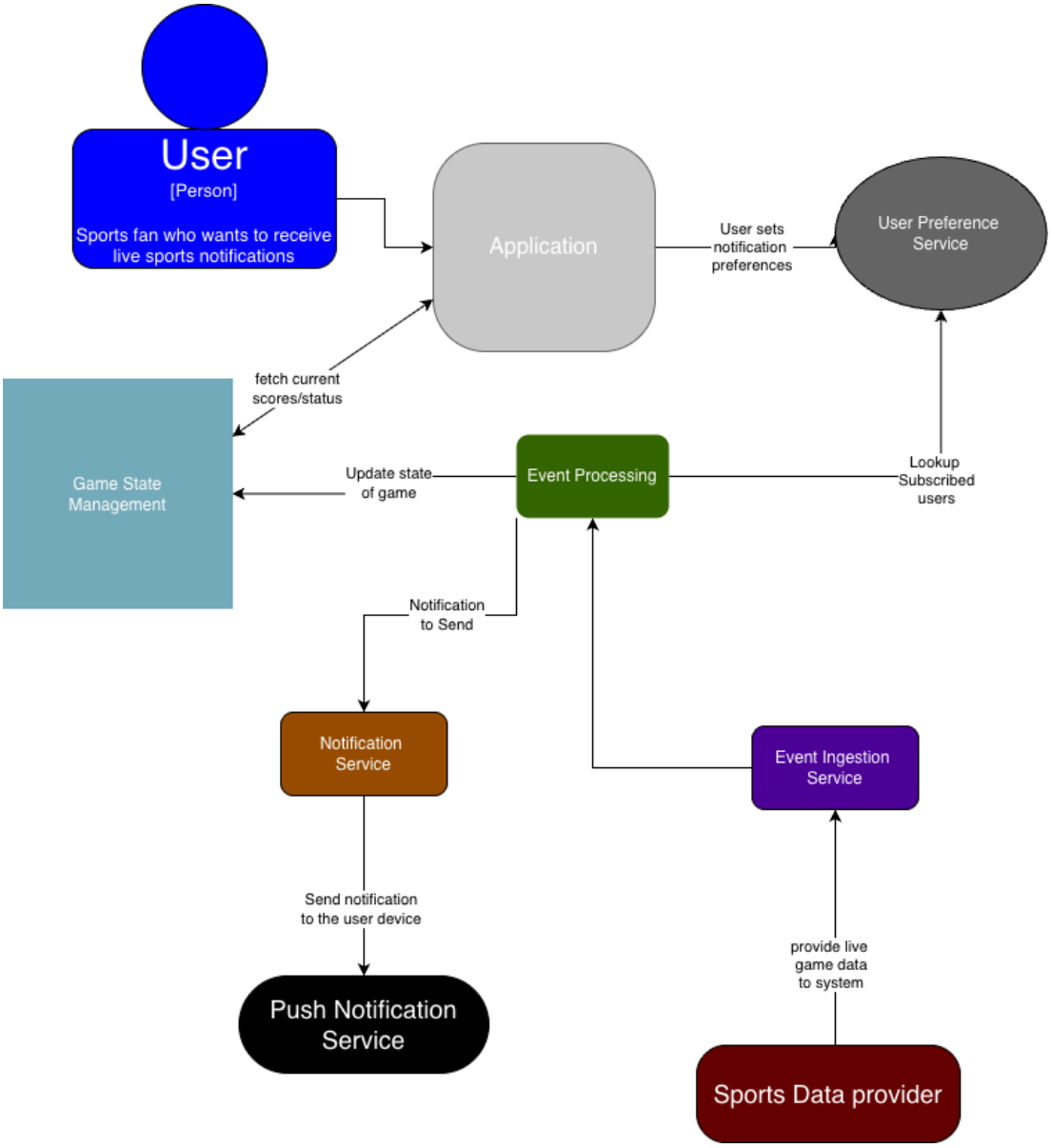


Figure 2 - **Container Diagram**



Stage 2: System design - high level

There are 3 main workflows

1) **User sets notification preferences** (User)
    a) User opens application
    b) User selects teams/games
    c) User toggles notifications
    d) App sends preferences to User Preference Service
    e) Preferences are stored
2) **Live event triggers notification** (Event driven)
    a) Sports Data Provider sends game event
    b) Event Ingestion Service receives event
    c) Event Processing Service processes event
    d) Event Processing queries User Preference Service for subscribers
    e) Game State Management updates game state
    f) Notification Service sends push notification
    g) Push Notification Service delivers notification to user device
3) **Use views live game information** (Read/view only)
    a) User opens app
    b) App requests game state
    c) Game State Management Service returns current score/status
    d) App displays game information

Component Features

Application (client)

- Create/manage user account
- Select team/games to follow
- Toggle alert types
- View notification history
- View live scores/updates

User Preference Service

- Store susbcriptions (maps teams/league/games to users)
- Returns subscribed users for an event
- User can update their preferences

Even Ingestion Service

- Receive events from sports provider
- Convert event data into proper format for system
- Forward events to Event Processing system
- If disconnected, try to connect back

Event Processing service

- Decides if an event should generate notifications
- Queries the USer Preference Service to get subscribers for a given event
- Updates game state
- Triggers the Notification Service

Game State Management Service

- Maintain most up to date score/status of games
- Stores recent/past event history
- Forwards game stat to application for user to see

Notification Service

- Build notification payload
- Send to Push notification service on user's device
- Write to the applications notification history

Java Public API skeleton

Models

- GameEvent
- UserPreferences
- GameState
- Notification

Services

- UserPreferenceService
    - void savePreferences(UserId user, UserPreferences prefs)
    - List<UserID> getSubscribers(GameEvent event)
    - UserPreferences getPreferences(UserId user)

- EventIngestionService
    - void onEvent(GameEvent event)

- EventProcessingService
    - void processEvent(GameEvent event)

- GameStateService
    - void updateGameState(GameEvent event)
    - GameState getGameState(GameId gameId)

- NotificationService
    - void sendNotification(List<UserId> users, Notification notification)
        - void sendNotification(UserId user, Notification notification)
    - List<Notification> getNotificationHistory(UserId user, int limit)

Diagram 3 - User Sets Notification Preferences BPMN

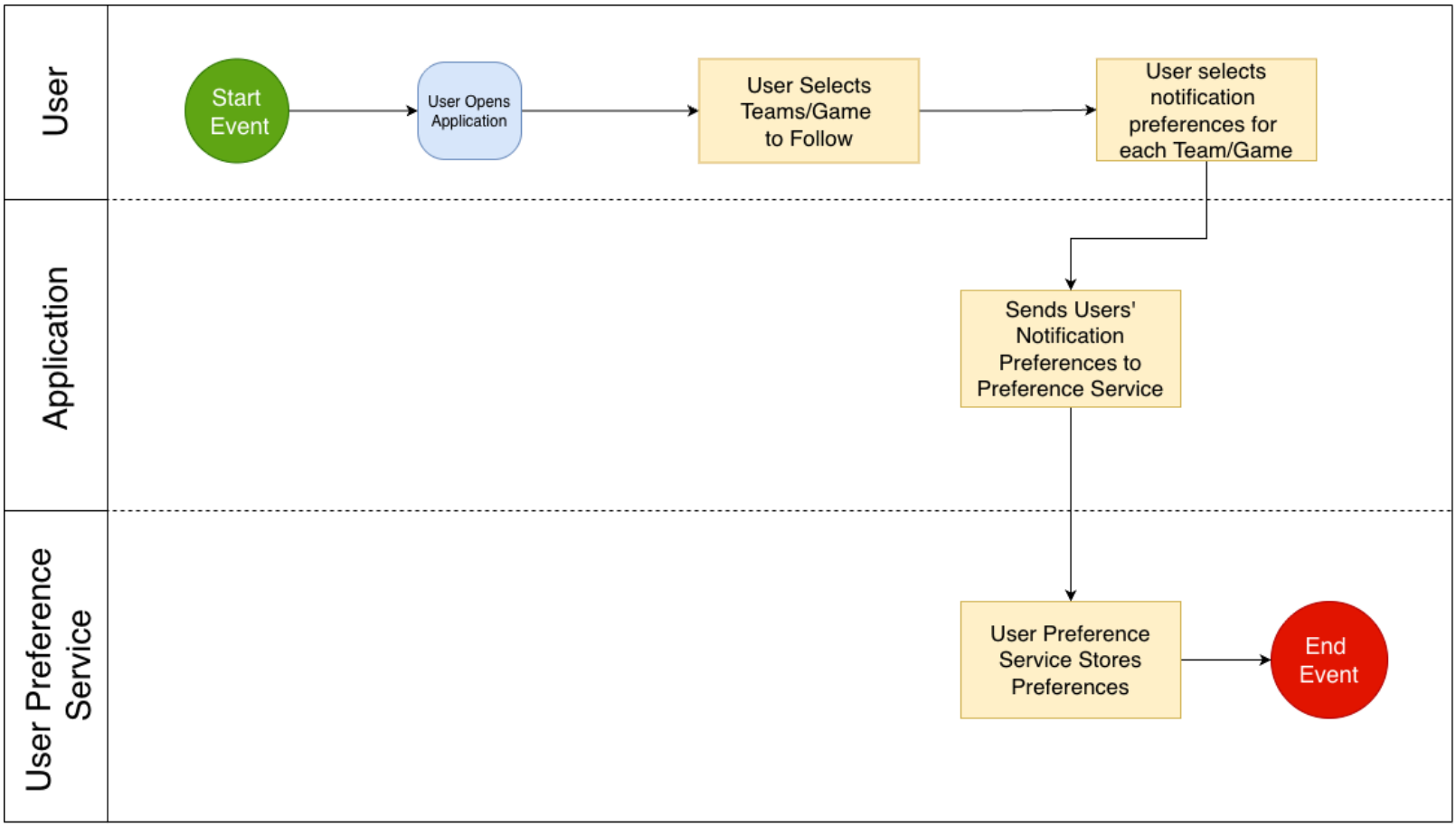


Diagram 4 - Live Event Notification BPMN

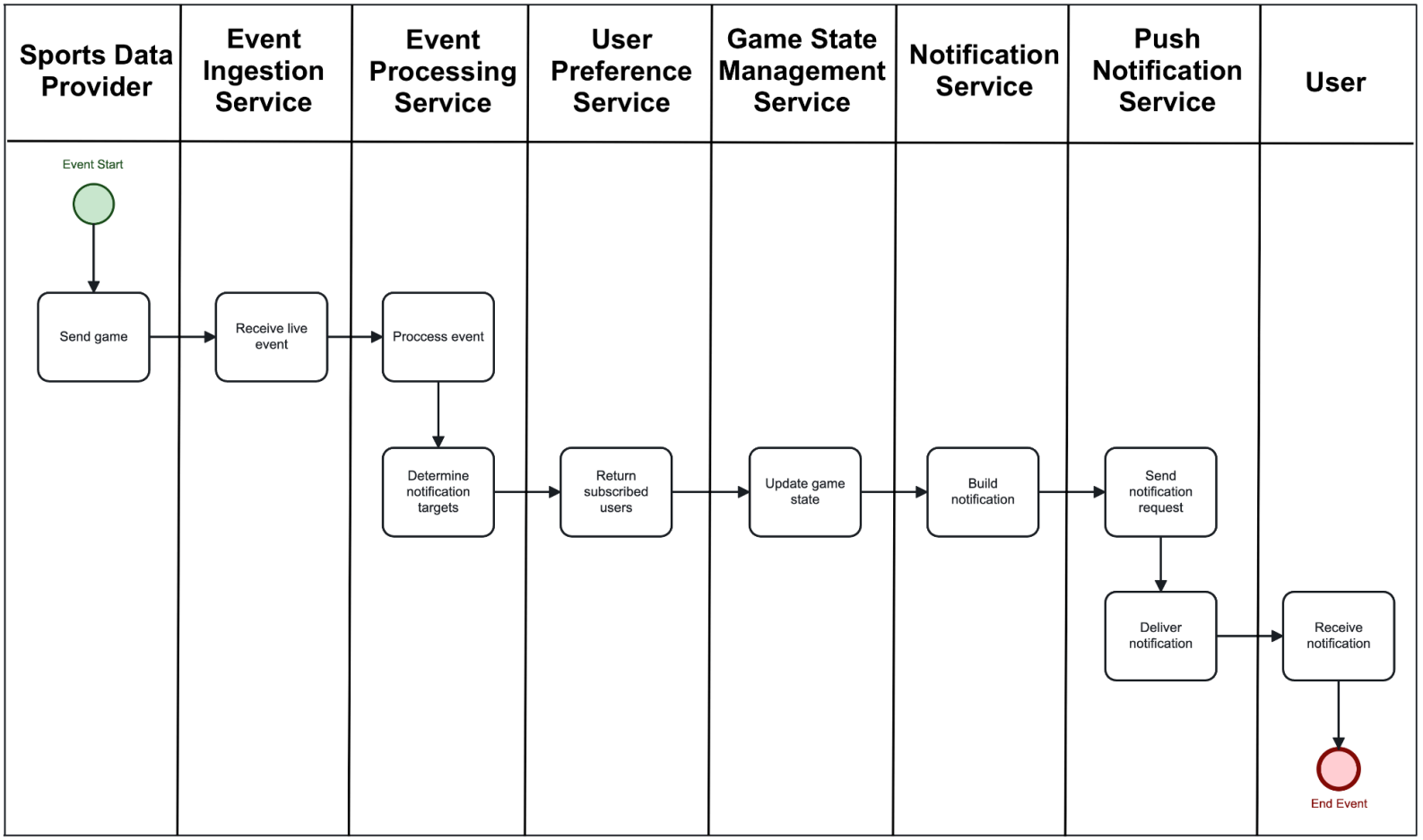


Diagram 5 - View Game State BPMN

